

1_GC에 대하여

1. GC는 무슨 일을 할까

애플리케이션의 동적 메모리 할당 요청을 자동으로 관리한다.

GC가 하는 일은 크게 네 가지로 나눌 수 있다.

- OS로부터 메모리를 할당받고 다시 돌려주는 일
- 애플리케이션이 요청할 때 그 메모리를 나눠주는 일
- 할당된 메모리 중 어느 부분이 사용 중인지 판별하는 일
- **더 이상 쓰이지 않는 메모리를 회수해 재사용하는 일**

이 중 GC의 핵심은 마지막, 회수(reclaim)다.

GC의 목적

GC의 목적은 개발자를 **수동 메모리 관리에서 해방**시키는 것이다. C/C++처럼 직접 할당하고 해제할 필요가 없으므로, 메모리 관련 오류에서 벗어날 수 있다.

다만 이는 **약간의 런타임 오버헤드를 대가로 얻는 것**이며, 100% 공짜는 아니다.

2. GC 선택이 중요한 이유 — 암달의 법칙

GC 선택이 항상 중요한 건 아니다. GC pause의 빈도와 지속 시간이 적당하다면, 별다른 튜닝 없이도 잘 돌아가는 애플리케이션이 많다.

하지만 **대용량 데이터, 많은 스레드, 높은 트랜잭션 처리율**을 가진 애플리케이션은 이야기가 다르다.

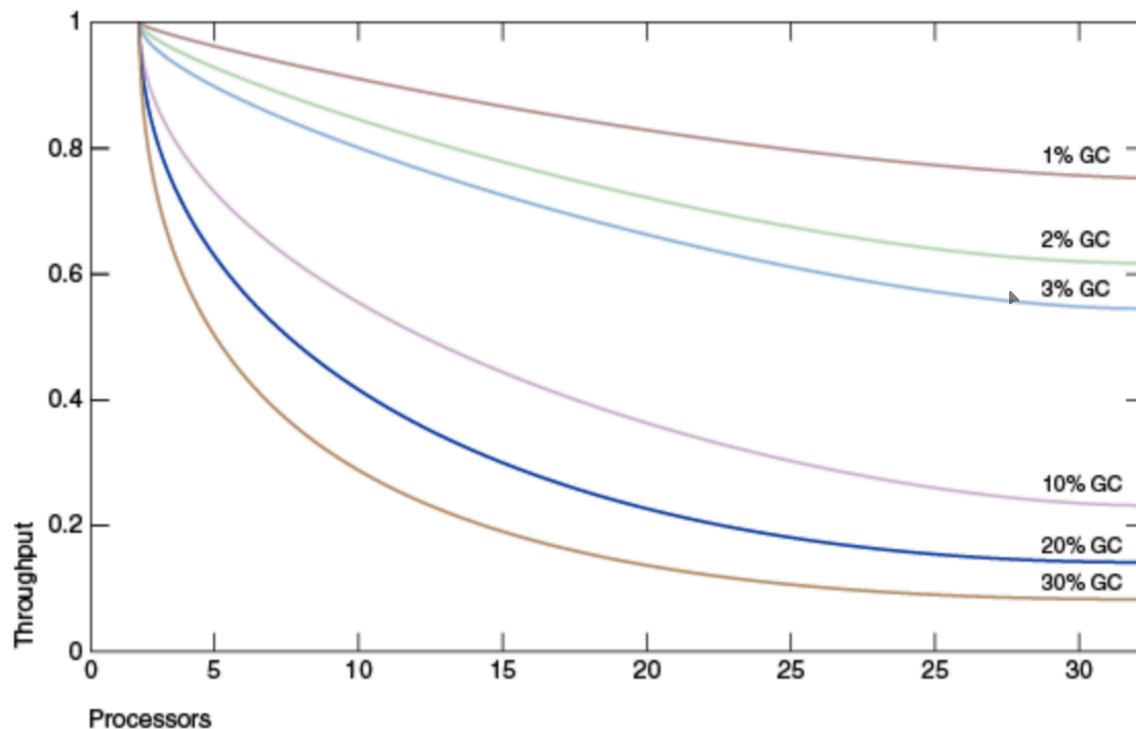
왜 달라질까

암달의 법칙: 병렬화로 얻는 속도 향상은 그 문제의 순차적인 부분에 의해 제한된다.

대부분의 작업은 완벽하게 병렬화될 수 없고, 항상 어떤 부분은 순차적으로 남아 병렬화의 이득을 받지 못한다. **GC 작업이 바로 이 순차적인 부분에 해당한다.** 애플리케이션의 병렬 작업이 아무리 빨라져도, GC의 순차적인 부분은 그만큼 빨라지지 않는다.

그래서 프로세서 수가 늘어날수록 GC 오버헤드가 미치는 영향은 오히려 커진다.

Figure 1-1 Comparing Percentage of Time Spent in Garbage Collection



- 단일 프로세서에서 GC에 **1%**의 시간만 쓰는 애플리케이션 → 32 프로세서에서는 처리량의 **20% 이상**을 잃는다.
- 단일 프로세서에서 GC에 **10%**의 시간을 쓰는 애플리케이션 → 32 프로세서에서는 처리량의 **75% 이상**을 잃는다.

작은 시스템에서 개발할 때는 무시할 만했던 문제가, 큰 시스템으로 확장하면 주요 병목이 될 수 있다. 역으로 말하면, **그 병목을 조금만 줄여도 성능에서 큰 이득을 얻을 수 있다.**

결국 충분히 큰 시스템에서는 올바른 GC를 선택하고, 필요하면 튜닝하는 것이 그만한 가치가 있다.

3. 결론

GC 오버헤드는 규모에 따라 다르게 작동한다.

HotSpot VM은 하나의 GC가 아니라 **여러 개의 GC를 제공한다.** Java SE는 애플리케이션이 실행되는 머신의 등급(class)을 판단해 가장 적절한 GC를 자동으로 선택한다. 다만 이 자동 선택이 모든 애플리케이션에 최적인 것은 아니다.

- **Serial 컬렉터**: 대부분의 소규모 애플리케이션(대략 100MB 힙)에 충분하다.
- **G1 컬렉터**: 대용량 메모리와 2개 이상의 프로세서를 가진 머신에서 돌아가는, 크고 스레드가 많은 애플리케이션의 기본값이다.

그 밖의 컬렉터들은 특화된 동작을 제공하는 대신, 추가적인 오버헤드나 복잡성을 감수해야 한다.